

Session 16 — Structures and Unions in C

Topics Covered

- What is a Structure?
- Need for Structures
- Declaration of a Structure
- Structure Variables
- Accessing Structure Members
- Initialization of Structures
- Arrays of Structures
- Structure vs Union

Learning Objectives

By the end of the session, students will be able to:

- Explain why structures are required in C.
- Create and declare a structure.
- Create structure variables.
- Access structure members using the . operator.
- Store multiple records using an array of structures.
- Differentiate between structures and unions.

1. What is a Structure?

A **structure** is a user-defined data type in C that allows us to group variables of **different data types** under one name.

For example, information about a student may contain:

Roll Number → int

Name → char[]

Marks → float

Instead of creating separate variables, we can group them into a structure.

Syntax

```
struct Student {  
    int rollNo;  
    char name[50];  
    float marks;  
};
```

Here:

- struct → keyword
- Student → structure name
- rollNo, name, marks → structure members

2. Declaring Structure Variables

Method 1

```
struct Student s1;
```

Method 2 — Declare variables along with structure

```
struct Student {  
    int rollNo;  
    char name[50];  
    float marks;  
} s1, s2;
```

3. Accessing Structure Members

The **dot (.) operator** is used to access structure members.

Example

```
#include <stdio.h>
```

```
struct Student {  
    int rollNo;  
    char name[50];  
    float marks;  
};
```

```
int main() {  
  
    struct Student s1;  
  
    s1.rollNo = 101;  
    s1.marks = 85.5;  
  
    printf("Roll No: %d\n", s1.rollNo);  
    printf("Marks: %.2f\n", s1.marks);  
  
    return 0;  
}
```

Output

Roll No: 101

Marks: 85.50

4. Structure Initialization

A structure can be initialized when its variable is declared.

```
#include <stdio.h>
```

```
struct Student {  
    int rollNo;  
    char name[30];
```

```
float marks;
};

int main() {

    struct Student s1 = {101, "Rahul", 89.5};

    printf("Roll No: %d\n", s1.rollNo);
    printf("Name: %s\n", s1.name);
    printf("Marks: %.2f\n", s1.marks);

    return 0;
}
```

Output

```
Roll No: 101
Name: Rahul
Marks: 89.50
```

5. Taking Input in a Structure

```
#include <stdio.h>
```

```
struct Student {
    int rollNo;
    char name[50];
    float marks;
};

int main() {

    struct Student s;

    printf("Enter roll number: ");
    scanf("%d", &s.rollNo);

    printf("Enter name: ");
    scanf("%49s", s.name);

    printf("Enter marks: ");
    scanf("%f", &s.marks);

    printf("\n--- Student Details ---\n");
```

```
printf("Roll No: %d\n", s.rollNo);
printf("Name: %s\n", s.name);
printf("Marks: %.2f\n", s.marks);

return 0;
}
```

6. Arrays of Structures

An **array of structures** is used when we need to store information about multiple objects/records.

For example:

Student 1

Student 2

Student 3

Student 4

Syntax

```
struct Student students[5];
```

Example

```
#include <stdio.h>
```

```
struct Student {
    int rollNo;
    char name[30];
    float marks;
};
```

```
int main() {
```

```
    struct Student students[3];
```

```
    for(int i = 0; i < 3; i++) {
```

```
        printf("\nEnter details of student %d\n", i + 1);
```

```
        printf("Roll No: ");
        scanf("%d", &students[i].rollNo);
```

```
        printf("Name: ");
        scanf("%29s", students[i].name);
```

```
        printf("Marks: ");
```

```

        scanf("%f", &students[i].marks);
    }

    printf("\n--- Student Records ---\n");

    for(int i = 0; i < 3; i++) {

        printf("\nRoll No: %d", students[i].rollNo);
        printf("\nName: %s", students[i].name);
        printf("\nMarks: %.2f\n", students[i].marks);
    }

    return 0;
}

```

Classroom Activity

Ask students to create a structure:

Employee

employeeId

name

salary

department

Then store details of **5 employees** using an array of structures.

7. Structure vs Union

Structure

Uses struct keyword

All members have separate memory

Multiple members can contain values simultaneously

Size is approximately the combined size of members, subject to padding

Useful for records

Structure Example

```

struct Data {
    int x;
    float y;
    char z;
};

```

All members can hold values simultaneously.

Union

Uses union keyword

Members share the same memory

Normally only one member's value is meaningful at a time

Size is based on the largest member, subject to alignment

Useful when different data representations share the same memory

Union Example

```
union Data {  
    int x;  
    float y;  
    char z;  
};
```

The members share the same memory location.

Demonstration Program

```
#include <stdio.h>
```

```
struct Student {  
    int roll;  
    float marks;  
};
```

```
union Data {  
    int roll;  
    float marks;  
};
```

```
int main() {  
  
    struct Student s;  
  
    s.roll = 101;  
    s.marks = 90.5;  
  
    printf("Structure:\n");  
    printf("Roll: %d\n", s.roll);  
    printf("Marks: %.2f\n", s.marks);  
  
    union Data d;  
  
    d.roll = 101;  
    printf("\nUnion Roll: %d\n", d.roll);  
  
    d.marks = 90.5;  
    printf("Union Marks: %.2f\n", d.marks);  
  
    return 0;  
}
```

Session 16 Assignment

Basic

1. Create a Book structure containing:
 - Book ID
 - Book name
 - Author
 - Price
2. Take input and display book details.

Intermediate

Create an array of structures to store **10 employee records**.

Display:

- All employees
- Employees having salary greater than ₹30,000
- Employee with the highest salary

Challenge

Create a **Student Result Management System** using an array of structures.

Store:

Roll No

Name

English

Maths

Science

Calculate:

Total

Percentage

Grade

Session 17 — File Handling in C

Topics Covered

- Introduction to File Handling
 - Why File Handling is Required
 - File Pointer
 - Opening a File
 - Writing to a File
 - Reading from a File
 - Appending to a File
 - Closing a File
 - fopen()
 - fclose()
-

Learning Objectives

Students will understand:

- Why files are required for permanent data storage.
 - What a file pointer is.
 - How to create and open files.
 - How to write data into files.
 - How to read data from files.
 - How to append data to existing files.
-

1. What is File Handling?

Normally, variables store data temporarily in memory.

When a program terminates, that data is lost.

File handling allows us to store data permanently in a file.

Without File Handling

Program



RAM



Program ends



Data lost

With File Handling

Program



File



Data remains stored

2. File Pointer

C uses a **file pointer** to work with files.

Syntax

```
FILE *fp;
```

FILE is a type provided by the C standard library.

3. Opening a File

The `fopen()` function is used to open a file.

Syntax

```
FILE *fp;
```

```
fp = fopen("filename.txt", "mode");
```

Example:

```
FILE *fp;
```

```
fp = fopen("data.txt", "w");
```

4. Closing a File

Use fclose() after finishing file operations.

```
fclose(fp);
```

5. Writing to a File

Example using fprintf()

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *fp;
```

```
    fp = fopen("student.txt", "w");
```

```
    if(fp == NULL) {
```

```
        printf("File cannot be opened.");
```

```
        return 1;
```

```
    }
```

```
    fprintf(fp, "Roll No: 101\n");
```

```
    fprintf(fp, "Name: Rahul\n");
```

```
    fprintf(fp, "Marks: 85.5\n");
```

```
    fclose(fp);
```

```
    printf("Data written successfully.");
```

```
    return 0;
```

```
}
```

Result

A file named:

student.txt

will contain:

Roll No: 101

Name: Rahul

Marks: 85.5

6. Reading from a File

```
#include <stdio.h>
```

```
int main() {  
  
    FILE *fp;  
    char ch;  
  
    fp = fopen("student.txt", "r");  
  
    if(fp == NULL) {  
        printf("File not found.");  
        return 1;  
    }  
  
    while((ch = fgetc(fp)) != EOF) {  
        printf("%c", ch);  
    }  
  
    fclose(fp);  
  
    return 0;  
}
```

Explanation

fgetc(fp)

reads one character from the file.

EOF

means **End Of File**.

7. Appending to an Existing File

Appending means adding new data **at the end** of an existing file without deleting the old data.

Use:

"a"

Example

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *fp;
```

```
fp = fopen("student.txt", "a");

if(fp == NULL) {
    printf("File cannot be opened.");
    return 1;
}

fprintf(fp, "\nRoll No: 102");
fprintf(fp, "\nName: Amit");
fprintf(fp, "\nMarks: 90");

fclose(fp);

printf("Data appended successfully.");

return 0;
}
```

8. Important Functions

Function Purpose

fopen() Opens a file
fclose() Closes a file
fgetc() Reads one character
fputc() Writes one character
fprintf() Writes formatted data
fscanf() Reads formatted data
fread() Reads binary/block data
fwrite() Writes binary/block data

Session 17 Mini Project — Student File System

Create a program that:

1. Takes student information.
2. Stores it in students.txt.
3. Reads the file.
4. Displays all records.
5. Allows adding another student without deleting existing records.

Example:

===== Student Management =====

1. Add Student
2. Display Students
3. Exit

Enter choice:

Session 17 Assignment

Basic

Create a file called:

message.txt

and write:

Welcome to C Programming

File Handling

into it.

Intermediate

Create a program to:

- Accept 5 student names.
- Store them in a file.
- Read the names.
- Display them.

Challenge

Create an **Employee File Management System**.

Store:

Employee ID

Name

Salary

Department

The program should support:

1. Add Employee
 2. Display Employees
 3. Add More Employees
 4. Exit
-

Session 18 — File Opening Modes and File I/O Functions

Topics Covered

- File Opening Modes
- Text and Binary Files
- r, w, a
- r+, w+, a+
- rb, wb
- rb+, wb+

- `getc()`
- `putc()`
- `getw()`
- `putw()`
- `fprintf()`
- `fscanf()`
- `fwrite()`
- `fread()`

1. File Opening Modes

Mode Meaning

<code>r</code>	Read existing text file
<code>w</code>	Write; creates/truncates file
<code>a</code>	Append; creates file if needed
<code>r+</code>	Read and write existing file
<code>w+</code>	Read and write; creates/truncates
<code>a+</code>	Read and append; creates if needed
<code>rb</code>	Read binary file
<code>wb</code>	Write binary file
<code>rb+</code>	Read/write binary existing file
<code>wb+</code>	Read/write binary; creates/truncates

Important Point

`w` is dangerous when an existing file contains important data because opening it in `w` mode **truncates the file**.

2. `r` Mode

```
FILE *fp = fopen("data.txt", "r");
```

Used to read an existing file.

If the file doesn't exist:

`fopen()` returns `NULL`

3. `w` Mode

```
FILE *fp = fopen("data.txt", "w");
```

Used for writing.

If the file doesn't exist:

New file → Created

If it already exists:

Old contents → Deleted

4. a Mode

```
FILE *fp = fopen("data.txt", "a");
```

New data is added at the end.

Old Data

+

New Data

5. r+

```
FILE *fp = fopen("data.txt", "r+");
```

Allows both:

Reading

Writing

The file must already exist.

6. w+

```
FILE *fp = fopen("data.txt", "w+");
```

Allows:

Reading + Writing

But existing contents are truncated.

7. a+

```
FILE *fp = fopen("data.txt", "a+");
```

Allows:

Reading + Appending

Existing data is preserved.

8. getc() / fgetc()

getc() reads one character from a stream.

```
int ch;
```

```
while((ch = getc(fp)) != EOF) {  
    printf("%c", ch);  
}
```

Using int for ch is important because EOF is an integer value that must be distinguishable from valid character values.

9. putc() / fputc()

Writes one character.

```
putc('A', fp);
```

Example

```
#include <stdio.h>
```

```
int main() {  
  
    FILE *fp;  
  
    fp = fopen("data.txt", "w");  
  
    if(fp == NULL)  
        return 1;  
  
    putc('H', fp);  
    putc('e', fp);  
    putc('l', fp);  
    putc('l', fp);  
    putc('o', fp);  
  
    fclose(fp);  
  
    return 0;  
}
```

The file contains:

Hello

10. fprintf()

Used to write formatted data.

```
fprintf(fp, "Name: %s\n", name);
```

Example:

```
#include <stdio.h>
```

```
int main() {  
  
    FILE *fp = fopen("employee.txt", "w");  
  
    if(fp == NULL)  
        return 1;  
  
    int id = 101;  
    char name[] = "Rahul";  
    float salary = 35000;  
  
    fprintf(fp, "%d %s %.2f", id, name, salary);
```

```
fclose(fp);

return 0;
}
```

11. fscanf()

Used to read formatted data from a file.

```
fscanf(fp, "%d %s %f", &id, name, &salary);
```

Example

```
#include <stdio.h>
```

```
int main() {

    FILE *fp = fopen("employee.txt", "r");

    if(fp == NULL)
        return 1;

    int id;
    char name[30];
    float salary;

    fscanf(fp, "%d %s %f", &id, name, &salary);

    printf("ID: %d\n", id);
    printf("Name: %s\n", name);
    printf("Salary: %.2f\n", salary);

    fclose(fp);

    return 0;
}
```

12. getw() and putw()

These functions are traditionally used for reading and writing integers in binary-oriented file operations, but they are **not part of the ISO C standard** and may not be available on every compiler.

Example on implementations that support them:

```
putw(100, fp);
```

Reading:

```
int n;
```

```
n = getw(fp);
```

For portable modern C programs, prefer standard functions such as `fread()` and `fwrite()` for binary data.

13. `fwrite()`

`fwrite()` is used to write binary data.

Syntax

```
fwrite(&variable, sizeof(variable), 1, fp);
```

Example

```
#include <stdio.h>
```

```
struct Student {  
    int roll;  
    char name[30];  
    float marks;  
};
```

```
int main() {  
  
    FILE *fp;  
  
    struct Student s = {101, "Rahul", 89.5};  
  
    fp = fopen("student.dat", "wb");  
  
    if(fp == NULL)  
        return 1;  
  
    fwrite(&s, sizeof(s), 1, fp);  
  
    fclose(fp);  
  
    printf("Student data written.");  
  
    return 0;  
}
```

14. `fread()`

Used to read binary data.

```
fread(&s, sizeof(s), 1, fp);
```

Complete Example

```
#include <stdio.h>
```

```
struct Student {  
    int roll;  
    char name[30];  
    float marks;  
};
```

```
int main() {
```

```
    FILE *fp;  
    struct Student s;
```

```
    fp = fopen("student.dat", "rb");
```

```
    if(fp == NULL) {  
        printf("File not found.");  
        return 1;  
    }
```

```
    if(fread(&s, sizeof(s), 1, fp) == 1) {  
        printf("Roll: %d\n", s.roll);  
        printf("Name: %s\n", s.name);  
        printf("Marks: %.2f\n", s.marks);  
    }
```

```
    fclose(fp);
```

```
    return 0;  
}
```

Text File vs Binary File

Text File	Binary File
Human-readable	Usually not human-readable
Stores textual representation	Stores bytes
Examples: .txt, .csv	Examples: .dat, images, executable data
fprintf(), fscanf() commonly used fwrite(), fread() commonly used	

Session 18 — Practical Activity

Activity: Employee Binary File System

Create:

```
struct Employee {  
    int id;  
    char name[30];  
    float salary;  
};
```

Store employee records using:

fwrite()

Then retrieve them using:

fread()

Students should enter **5 employees** and display all records.

Session 18 Mini Project — Employee Record Management

Build a menu-driven application:

```
=====
EMPLOYEE MANAGEMENT
=====
```

1. Add Employee
2. Display Employees
3. Search Employee
4. Exit

Enter choice:

Employee Structure

```
struct Employee {  
    int id;  
    char name[50];  
    float salary;  
};
```

Requirements

- Store records in employees.dat.
- Use fwrite() for storing.
- Use fread() for retrieving.
- Search employee by ID.
- Do not lose existing records when adding new employees.
- Use appropriate file opening modes.

Session 18 Assignment

Basic

Write programs demonstrating:

1. r
2. w
3. a

Explain what happens to existing data in each mode.

Intermediate

Create a program that:

- Writes 5 integers into a binary file using fwrite().
- Reads them using fread().
- Calculates their sum and average.

Advanced

Create a **Student Record Management System** using structures and binary files.

Features:

1. Add Student
2. Display All Students
3. Search Student
4. Update Student
5. Exit

Student data:

Roll Number

Name

Marks

Percentage

Grade

Use:

struct

fopen()

fclose()

fread()

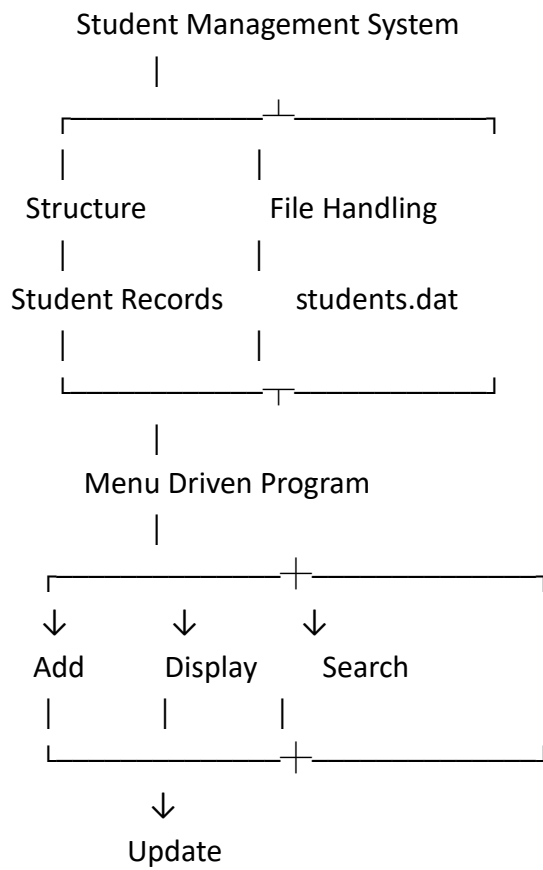
fwrite()

Overall Practical Progression

Session	Main Concept	Practical
16	Structures & Unions	Student/Employee Records
17	Basic File Handling	Text File Management
18	File Modes & Advanced I/O	Binary Employee Management

Final Combined Project

At the end of Session 18, students can combine all three sessions to create a **Student Management System**:



This gives students a natural progression from **structures** → **records** → **files** → **persistent record-management applications**.